

< Previous	 	 	 	 	 		Next >
------------	---	---	---	---	---	---	--------

6. Allow the rider to move along the zipline

 Bookmark this page

Video note: There is a misprint in the equation for v_y at the beginning of this video: the term $-\sin(\theta_R)$ should be $+\sin(\theta_R)$.

Understanding the code generally



mathematical model,
we'll use it to understand the
complicated behaviors of a zip
line.
You'll begin by using the MATLAB code
to investigate
how the weight of a rider affects
the shape of their trajectory and the
time
it takes for them to reach the end.
You'll then investigate how other
parameters,
such as the vertical drop and the slack,
affect the trajectory of a rider.
And finally, you'll design a zip line of
your own.



Video

 [Download video file](#)

Transcripts

 [Download SubRip\(.srt\) file](#)
 [Download Text\(.txt\) file](#)

Note on video: The code we are providing you actually uses a Runge-Kutta 4th order method rather than ODE45. This was to optimize the plotting of the solution, but the underlying principles are the same.

We have written some MATLAB code that models a moving rider along a zipline. Here are instructions for downloading and running the code.

1. Login into [MATLAB Online](#) using your [license](#) from this course (or use a desktop version R2016b or later).
2. Navigate the Current Folder browser to where you want to save the file.
3. Copy and paste the following code in the MATLAB to download the scripts.

```
url = 'https://courses.edx.org/asset-v1:MITx+18.033x+1T2018+type@asset+block@plotCat.m';  
websave('plotCat.m', url);  
url = 'https://courses.edx.org/asset-v1:MITx+18.033x+1T2018+type@asset+block@zipRHS.m';  
websave('zipRHS.m', url);  
url = 'https://courses.edx.org/asset-v1:MITx+18.033x+1T2018+type@asset+block@startPos.m';  
websave('startPos.m', url);  
url = 'https://courses.edx.org/asset-v1:MITx+18.033x+1T2018+type@asset+block@catSys.m';  
websave('catSys.m', url);  
url = 'https://courses.edx.org/asset-v1:MITx+18.033x+1T2018+type@asset+block@zipsys.m';  
websave('zipsys.m', url);
```

4. Open the **zipsys.m** file. Try running this code for a zipline rider that weighs **1kg** and one that weighs **100kg**. (You will need to edit the parameter **p.m** in the zipsys file.) Observe that when the mass of the rider is very small, the trajectory is very close to the catenary. And when the mass is large, it approaches the ellipse.

The output of this code will be two graphs. The upper plot shows a parametric plot of the trajectory of the rider in blue. The lower plot shows the velocity of the rider as a function of time.

The code is written to stop running once the position of the rider reaches **450**, or the time step is larger than **100**.

Acknowledgements

Thanks are due to Timothy Hall for asking David Jerison the question, "how do ziplines behave mathematically?" We also thank David Custer and Susan Ruff who helped with real life ziplines and shared MIT student experiments on ziplines. Thanks to Prof. Dan Frye for letting us film Miles Couchman and Sam Turton on the zipline in his backyard.

Big thanks to Prof. Ken Kamrin for writing the original MATLAB code modeling the zipline, and Miles Couchman for designing this project around a simplified model.

6. Allow the rider to move along the zipline

Hide Discussion

Topic: Final project: Applications to nonlinear differential equations / 6. Allow the rider to move along the zipline

Add a Post

Show all posts ▼by recent activity ▼

There are no posts in this topic yet.

✕

◀ Previous

Next ▶